CSS Complete Notes

Selectors • Specificity • Cascade • Box Model • Sizing Units • Anchor Positioning • Flexbox • Grid

Roman Urdu mein — Momin ke liye

Contents

1. CSS Selectors

CSS selector woh cheez hai jo batati hai ke HTML ke kis element (ya elements) par style apply karni hai. Ek CSS **rule** do parts se milta hai: **selector** + **declaration block** (curly braces `{}` ke andar property-value pairs).

```
.my-css-rule {
  color: red;
  font-size: 1.5em;
}
```

Yahan `.my-css-rule` selector hai, aur andar declarations hain.

Simple Selectors

Universal Selector (*)

`*` — page ke har element ko match karta hai (wildcard).

```
* { color: hotpink; }
```

Type Selector

Directly HTML tag ko target karta hai.

```
section { padding: 2em; }
```

Class Selector (.)

HTML element ke `class` attribute ko match karta hai. Ek element ke multiple classes ho sakti hain, aur CSS sirf yeh dekhta hai ke class attribute mein woh naam *contain* hota hai ya nahi — exact match zaroori nahi.

```
.my-class { color: red; }

<div class="my-class another-class"></div>
```

Yaad rahe: Class ya ID number se shuru nahi ho sakti — jaise `.1element` invalid hai.

ID Selector (#)

Har page par ID sirf ek element par honi chahiye (unique). Isliye ID selectors ko overuse karne se reusability kam ho jati hai.

```
#rad { border: 1px solid blue; }
```

Attribute Selector ([ ])

HTML attribute ki presence ya value ke basis par select karta hai.

```
[data-type='primary'] { color: red; }   /* exact value */
[data-type] { color: red; }             /* sirf attribute ki presence */
[href*='example.com'] { color: red; }   /* contains */
[href^='https'] { color: green; }       /* starts with */
[href$='.com'] { color: blue; }         /* ends with */
[data-type='primary' s] { }             /* case-sensitive (s operator) */
[data-type='primary' i] { }             /* case-insensitive (i operator) */
```

Grouping Selectors

Comma se multiple selectors ko ek saath group kar sakte hain.

```
strong, em, .my-class, [lang] {
  color: red;
}
```

Pseudo-classes aur Pseudo-elements

Pseudo-classes (single colon :)

Element ki **state** ko target karte hain — jaise hover hona, ya nth child hona.

```
a:hover { outline: 1px dotted green; }
p:nth-child(even) { background: floralwhite; }
```

Pseudo-elements (double colon ::)

Ek naya "virtual" element insert karte hain ya element ke kisi specific part ko target karte hain.

```
.my-element::before { content: 'Prefix - '; }
li::marker { color: red; }
::selection { background: black; color: white; }
```

Note: Purane browsers (jaise IE8) ke sath backward-compatibility ke liye single colon (`:before`) bhi kaam karta hai, lekin standard double colon hai.

Complex Selectors (Combinators)

Combinator woh symbol hai jo do selectors ke darmiyan hota hai — element ki document position ke basis par select karne mein madad karta hai. **Ek baat yaad rahe:** combinators sirf neeche (child) ki taraf select karte hain, upar (parent) ki taraf nahi.

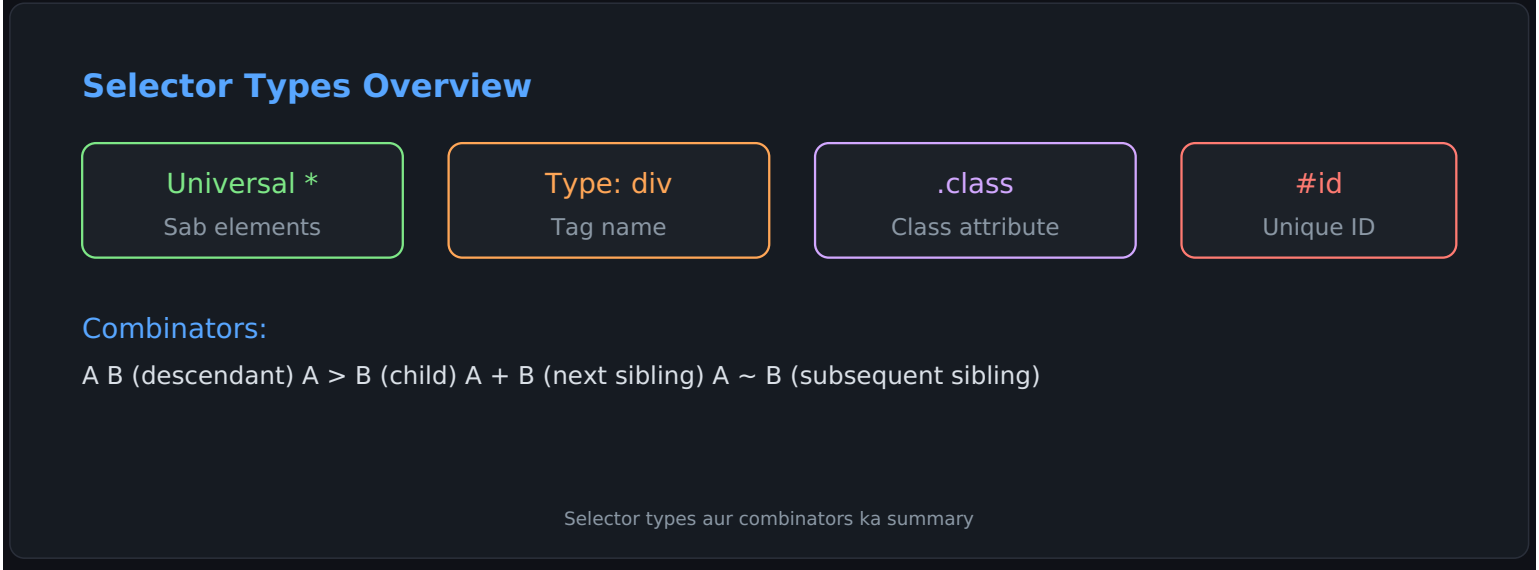
Combinator	Symbol	Kaam
Descendant	space	Har level ke child elements (recursive)
Next sibling	+	Immediate agla sibling
Subsequent sibling	~	Koi bhi baad wala sibling (same parent)
Child (direct descendant)	>	Sirf direct children

```
p strong { color: blue; }        /* descendant - recursive */
.top * + * { margin-top: 1em; }  /* next sibling + universal */
.top > * + * { }                 /* direct children only */
input:checked ~ .switch { }      /* subsequent sibling */
```

Compound Selectors

Multiple selectors ko bina combinator ke jodna, specificity aur precision badhata hai.

```
a.my-class { color: red; }  /* sirf woh <a> jis par .my-class bhi ho */
```



Selector types aur combinators ka summary

2. Specificity

Jab do CSS rules same element ko target karti hain aur unke declarations mein conflict ho, to browser **specificity algorithm** se decide karta hai konsi declaration jeetegi.

```
<button class="branding">Hello</button>

.branding { color: blue; }
button    { color: red; }
/* .branding zyada specific hai, isliye button BLUE hoga */
```

(A, B, C) Triad

Specificity decimal number nahi, balke teen values ka triad hai: **A-B-C**

- **A** — ID-like specificity (ID selectors)
- **B** — Class-like specificity (class, pseudo-class, attribute)
- **C** — Element-like specificity (type, pseudo-element)

Comparison Rule

Pehle A compare hota hai; agar barabar ho to B; phir C. Jo pehle bada A rakhta hai, woh jeet jata hai — chahe B ya C kam bhi ho.

```
(1,0,0) is bigger than (0,4,3)  /* kyun ke A: 1 > 0 */
```

Scoring Table

Selector	Specificity
`*` (universal)	(0,0,0)
`div`, `::selection`	(0,0,1)
`.my-class`, `:hover`, `[href]`	(0,1,0)
`#myID`	(1,0,0)
`a[href="#"]`	(0,1,1)
`section.awesome`	(0,1,1)

Special Cases

- `:not()` aur `:is()` khud kuch add nahi karte, lekin unke andar sabse specific argument ka specificity le lete hain.
- `:where()` hamesha **(0,0,0)** hota hai — chahe andar ID ho.

```
:is(h1, h2, h3) { }              /* (0,0,1) - type ka max */
:is(h1, h2, h3, #my-heading) { } /* (1,0,0) - ID zyada specific */
:where(#my-content) { }          /* hamesha (0,0,0) */
```

Factors jo Specificity Affect NAHI karte

- **Inline style attribute** — specificity ka part nahi, yeh cascade ke earlier step mein evaluate hota hai (lekin phir bhi normal rules ko override karta hai).
- `!important` — specificity nahi badhata, balke declaration ko alag "origin" mein bhej deta hai.

Specificity ko Pragmatically Boost Karna

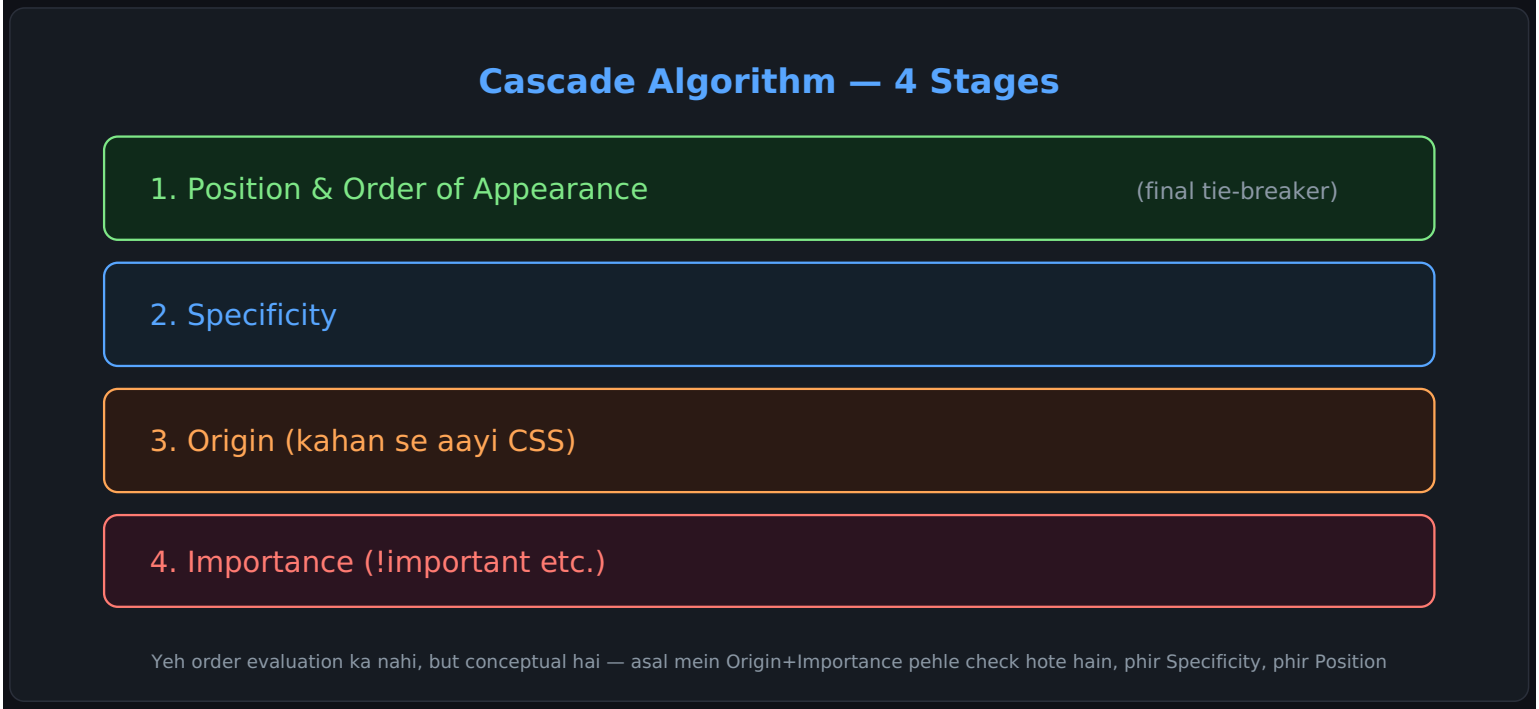
Agar zaroorat pade to class selector ko repeat kar ke specificity barha sakte hain:

```
.my-button.my-button { background: blue; } /* (0,2,0) */
```

Caution: Agar baar baar aisa karna par raha hai, matlab CSS bohat zyada specific likhi ja rahi hai. Better hai selectors ko simple rakhein aur specificity ko sirf zaroorat ke mutabiq use karein.

3. The Cascade

CSS ka full form hi **Cascading Style Sheets** hai. Cascade woh algorithm hai jo conflicting CSS rules ke darmiyan decide karta hai konsi declaration jeetegi. Iske 4 stages hain, is order mein evaluate hote hain:



1. Position aur Order of Appearance

Jab sab kuch (specificity, origin, importance) barabar ho, to jo rule **baad mein** declare hui ho, woh jeet jati hai.

```
.my-element {
  background: green;
  background: purple; /* yeh jeetega, baad mein hai */
}
```

Yeh technique fallback values ke liye bhi useful hai — agar browser `clamp()` support nahi karta to purani value use hogi:

```
.my-element {
  font-size: 1.5rem;
  font-size: clamp(1.5rem, 1rem + 3vw, 2rem);
}
```

2. Specificity

(Poori detail Topic 2 mein cover ho chuki hai.) Zyada specific selector jeet jata hai, chahe woh pehle likha gaya ho.

3. Origin

CSS sirf aapki authored nahi hoti — browser, OS, extensions se bhi aati hai. Origin ka order (kam se zyada specific):

1. User agent base styles (browser defaults)
2. Local user styles (OS/extension level)
3. Authored CSS (aapki likhi hui)
4. Authored `!important`
5. Local user styles `!important`
6. User agent `!important`

4. Importance

Order of importance (kam se zyada):

1. Normal rule (color, background, etc.)

2. `animation` rule type
3. `!important` rule type
4. `transition` rule type (sabse zyada important — active state mein)

> **DevTools Tip:** Browser DevTools mein saari matching CSS dikhti hai — jo apply nahi ho rahi woh strikethrough (crossed-out) hoti hai. Agar expected CSS wahan bilkul dikh hi nahi rahi, to selector mein typo ya invalid CSS check karein.

4. Inheritance

Kabhi kabhi CSS likhte waqt ek property "khud ba khud" apply ho jati hai jo aap ne set hi nahi ki thi. Yeh **inheritance** ki wajah se hota hai — kuch CSS properties agar explicitly set na ki jayein, to woh apne **parent element** se value le leti hain.

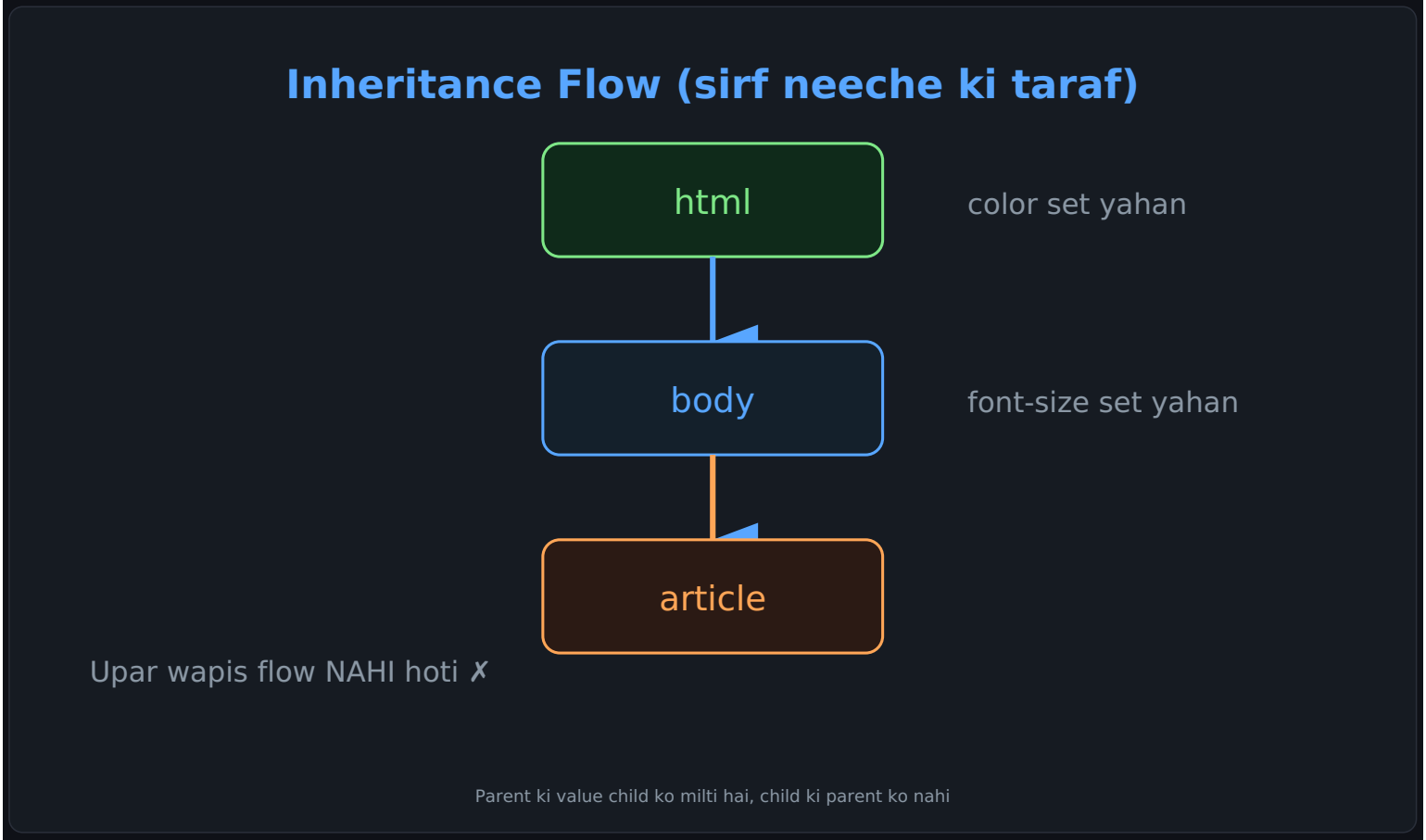
```
article a { color: maroon; }

<article>
  <a href="#" class="my-button">I am a button link</a>
</article>
/* .my-button ka text color maroon hoga, kyun ke color
   inherited property hai aur article se inherit hui */
```

Inheritance Kaise Flow Karti Hai

Inheritance hamesha **neeche (parent → child)** ki taraf flow karti hai — kabhi upar nahi. Root element (`<html>`) kuch inherit nahi karta kyun ke woh document ka sabse pehla element hai.

```
html { color: lightslategray; }
body { font-size: 1.2em; }
p    { font-style: italic; }
```

Is example mein: `color` saare descendants ko milega. `font-size` sirf body ke andar wale elements ko milega — `html` khud apna default size rakhega (kyun ke inheritance sirf neeche jati hai, upar nahi). `font-style: italic` sirf `<p>` par hoga, kyun ke woh sabse andar (deepest nested) hai.



Konsi Properties Default Inherit Hoti Hain?

Sab properties inherit nahi hotin — sirf kuch specific ones. Zyada tar yeh **text/typography** se related properties hain:

Category	Properties
Text/Font	color, font-family, font-size, font-style, font-variant, font-weight, font, letter-spacing, line-height, text-align, text-indent, text-transform, word-spacing, white-space
List	list-style, list-style-image, list-style-position, list-style-type

Table	border-collapse, border-spacing, caption-side, empty-cells
Others	cursor, direction, orphans, widows, quotes, visibility

> **Yaad rakhne ka tareeqa:** Zyada tar typography-related properties inherit hoti hain. Layout properties (jaise `margin`, `padding`, `width`, `border`, `display`) **inherit NAHI** hotin.

Har Property ki ek "Initial Value" Hoti Hai

Har HTML element ki har CSS property ka ek default **initial value** hota hai. Agar cascade kisi property ki value calculate na kar paye, to yeh initial value use hoti hai — chahe woh property inherited ho ya na ho.

Inheritance ko Control Karne Wale Keywords

1. `inherit`

Kisi bhi property (chahe woh default inherit na hoti ho) ko force se parent ki computed value lene par majboor karta hai. Yeh exceptions banane ke liye useful hai.

```
strong { font-weight: 900; }
.my-component { font-weight: 500; }

/* .my-component ke andar strong bhi 500 chahiye: */
.my-component strong { font-weight: inherit; }
```

> **Faida:** Agar future mein `.my-component` ki value change ho, `strong` automatically sync rahega — kyun ke hardcoded value ki bajaye `inherit` use ki.

2. `initial`

Property ko wapis uski **default/initial** value par reset karta hai (inheritance ignore ho jati hai).

```
aside strong { font-weight: initial; }
/* aside ke andar strong ka bold hatt jayega, default weight aa jayegi */
```

3. `unset`

Yeh smart keyword hai — property ke behavior par depend karta hai:

- Agar property **default inherit** hoti hai → `unset` = `inherit` ki tarah kaam karega
- Agar property **default inherit nahi** hoti → `unset` = `initial` ki tarah kaam karega

```
aside p {
  margin: unset; /* margin inherit nahi hoti -> initial jaisa (0) */
  color: unset;  /* color inherit hoti hai -> parent ki value le legi */
}
```

> **Kab Useful:** Jab yaad na ho ke koi property inherit hoti hai ya nahi, `unset` hamesha "sahi" cheez karega.

`all` property ke sath combine kar ke saari properties ek sath reset kar sakte hain — future mein naye properties add hon to bhi automatically cover ho jati hain:

```
aside p { all: unset; }
```

4. `revert`

Sirf usi **origin** (jahan revert likha hai) ke styles ko undo karta hai, aur ek origin peeche chala jata hai (user preference ya user-agent default).

```
p { padding: 2em; }
aside p { padding: revert; } /* authored padding hat jayega,
                                user-agent default aa jayega */
```

> **Fark `unset` aur `revert` mein:** `inherit` / `initial` / `unset` yeh specify karte hain ke value *kya* honi chahiye. `revert` sirf itna kehta hai ke

> "mere likhe hue styles yahan apply mat karo" — baaki origins (user-agent, user preference) apna kaam karte rahenge.

5. `revert-layer`

Cascade layers ke context mein — `revert` jaisa hi hai, lekin sirf **current layer** ke styles ko hide karta hai, poore author origin ko nahi. Third-party library ko layer mein import kar ke apne overrides upar layer mein rakhna common pattern hai — `revert-layer` se override hata kar library ka default wapis mil jata hai.

Quick Comparison

Keyword	Behavior
`inherit`	Hamesha parent ki computed value le leta hai
`initial`	Hamesha property ki default/initial value par reset
`unset`	Inherited property ho to inherit, warna initial
`revert`	Sirf current origin ke authored styles hatata hai, ek origin peeche chala jata hai
`revert-layer`	Sirf current cascade layer ke styles hatata hai

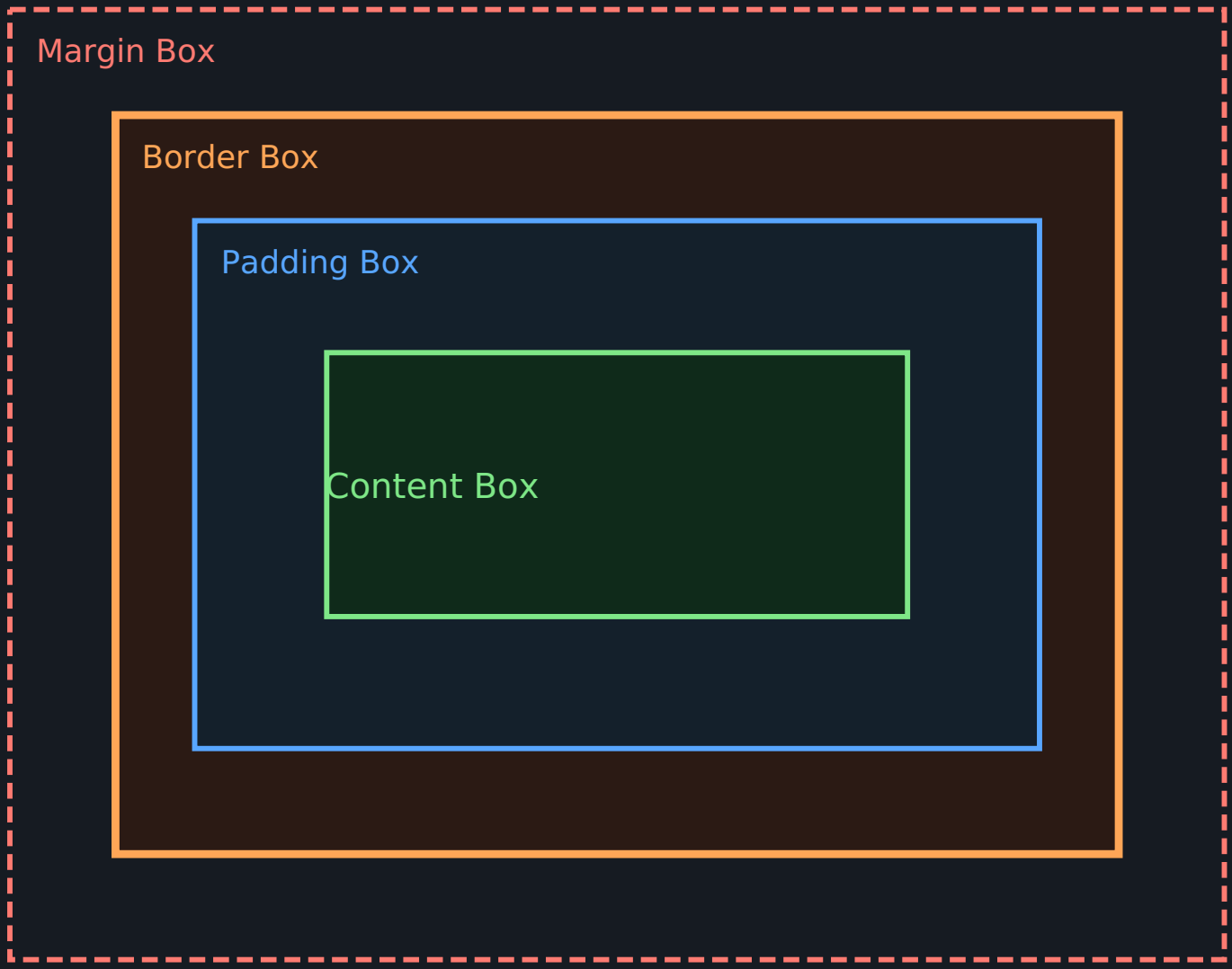
5. The Box Model

CSS mein **har cheez ek box hoti hai** — chahe woh sirf text ho ya `border-radius` se circle jaisa dikh raha element ho. Box model samajhna is liye zaroori hai ke content apni box mein sahi fit ho, overflow na ho.

Extrinsic vs Intrinsic Sizing

- **Extrinsic sizing** — aap khud width/height define karte hain (fixed control, lekin overflow ka risk).
- **Intrinsic sizing** — browser content ke hisaab se size decide karta hai (jaise `width: min-content`) — zyada flexible, overflow kam.

Box Model ke 4 Areas



Content → Padding → Border → Margin (andar se bahar)

Area	Kaam
Content Box	Actual content (text/images) yahan hota hai — content parent ka size control kar sakta hai
Padding Box	Content ke ird-gird ka space — background yahan visible rehta hai, scrollbars bhi yahan
Border Box	Visual frame — **border edge** waha tak hai jo dikhta hai
Margin Box	Box ke bahar ka space — outline aur box-shadow yahan "paint" hote hain, box size nahi badhate

Analogy — Picture Frame

- Content box = artwork

- Padding box = white mounting board
- Border box = frame
- Margin box = frames ke darmiyan space

box-sizing Property

Default browser behavior: `box-sizing: content-box`. Iska matlab jo `width` / `height` set karte hain woh sirf content box par apply hoti hai — padding aur border **add** ho jate hain.

```
.my-box {
  width: 200px;
  border: 10px solid;
  padding: 20px;
}
/* Actual width = 200 + 40(padding) + 20(border) = 260px */
```

Isko fix karne ke liye `border-box` use karte hain — ab width/height **border box** par apply hoti hai, padding/border andar "push" ho jate hain:

```
.my-box {
  box-sizing: border-box;
  width: 200px;   /* actual rendered width bhi 200px hi rahegi */
  border: 10px solid;
  padding: 20px;
}
```

Best Practice — Universal Reset:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Yeh CSS resets/normalizers mein bohat common hai kyun ke yeh predictable sizing deta hai.

Display Defaults

Element	Default display
`<div>`	block
`<li>`	list-item
`<span>`	inline

- **block** — available inline space fill karta hai (poori width)
- **inline** — sirf content jitna size, block margins doosre elements ignore karte hain
- **inline-block** — inline ki tarah lekin doosre elements block margin respect karte hain

6. Sizing Units

Numbers

Bina unit ke numbers (unitless) — jaise `opacity`, `line-height`, ya rgb ke color channels.

```
p { font-size: 24px; line-height: 1.5; } /* 1.5 = 150% of font-size = 36px */
```

Best Practice: `line-height` ko hamesha unitless value dein (jaise `1.5`), fixed px nahi — kyun ke `font-size` inherit hoti hai, aur unitless line-height us ke sath scale hoti hai.

Percentages

`width` ka percentage parent element ki available width se calculate hota hai. `margin` aur `padding` percentage mein ho to woh (kisi bhi direction mein) parent ki **width** se calculate hote hain.

```
div { width: 300px; height: 100px; }
div p {
  width: 50%;         /* = 150px */
  margin-top: 50%;    /* = 150px (parent width se) */
  padding-left: 50%;  /* = 150px */
}
```

Absolute Lengths

Yeh hamesha ek hi fixed base se calculate hoti hain — predictable. Print ke liye best hain (screens par physical accuracy guaranteed nahi).

Unit	Name	Equivalent
cm	Centimeters	1cm = 96px/2.54
mm	Millimeters	1/10 of 1cm
in	Inches	1in = 2.54cm = 96px
pt	Points	1/72 of 1in
px	Pixels	1/96 of 1in

Relative Lengths — Font-size Based

Unit	Relative to
`em`	Parent ka font-size (1.5em = 50% larger)
`rem`	Root (html) element ka font-size (default 16px)
`ch`	"0" character ki average width
`ex`	Roughly x-height ya .5em
`cap`	Capital letters ki height
`lh`	Element ka line-height

Viewport-Relative Units

Unit	Relative to
`vw`	1% viewport width
`vh`	1% viewport height
`vmin`	1% chota dimension

`vmax`	1% bada dimension

Mobile browsers ke UI show/hide hone ki wajah se alternative units bhi hain: `lvh` (large viewport), `svh` (small viewport), `dvh` (dynamic viewport — real-time change hoti hai).

Container-Relative Units

Container queries ke andar useful — `cqw`, `cqh`, `cqi`, `cqb`, `cqmin`, `cqmax` — 1% of container ka width/height.

> **Objective:** Text ke liye `px` ki bajaye `em` / `rem` use karein — is se system-level font size preference honor hoti hai (accessibility ke liye behtar).

7. Anchor Positioning

Pehle tooltip/dropdown ko kisi element ke relative position karne ke liye JavaScript ya complex absolute positioning use karni parti thi. CSS Anchor Positioning ab declaratively yeh kaam karti hai.

Anchor Banane ka Tareeqa

```
#anchor {
  anchor-name: --my-anchor;   /* naam do dashes se start hota hai */
}

#positionedElement {
  position: absolute;             /* ya fixed */
  position-anchor: --my-anchor;   /* kis anchor se tether karna hai */
  position-area: end;             /* kahan position karna hai */
}
```

Implicit Anchors (Popovers)

Popover ka anchor already implicit set hota hai (jab `popovertarget` se open kiya jaye). Sirf `position-area` set karna kaafi hai — margin ko `unset` karna zaroori hai kyun ke popover ka default margin `auto` hota hai.

anchor-scope

Agar same `anchor-name` multiple jagah repeat ho (jaise ek list ke andar), to `anchor-scope` se ensure karte hain ke har positioned element sirf apne nearby anchor se match ho — na ke document ke last wale anchor se.

position-area Keywords

Type	Keywords
Physical	top, left, bottom, right, center
Logical	block-start, block-end, inline-start, inline-end
Span	span-top, span-inline-end, span-all

```
position-area: block-end span-inline-end; /* common dropdown pattern */
```

anchor() Function

Advanced control ke liye — individual inset properties set karta hai based on anchor ki position. Yeh ek CSS length return karta hai, isliye `calc()` mein bhi use ho sakta hai.

```
.positionedElement {
  block-start: anchor(--my-anchor start);
  block-start: anchor(--my-anchor, 100px); /* fallback agar anchor na mile */
}
```

Overflow Handling — Fallbacks

`position-try-fallbacks` se multiple fallback positions define kar sakte hain — jo overflow na ho, browser wahi use karega.

```
.positioned-element {
  position-area: block-end span-inline-end;
  position-try-fallbacks: block-end span-inline-start;
}
```

Common keywords: `flip-block`, `flip-inline`, `flip-start`. Custom fallback ke liye `@position-try` use karte hain.

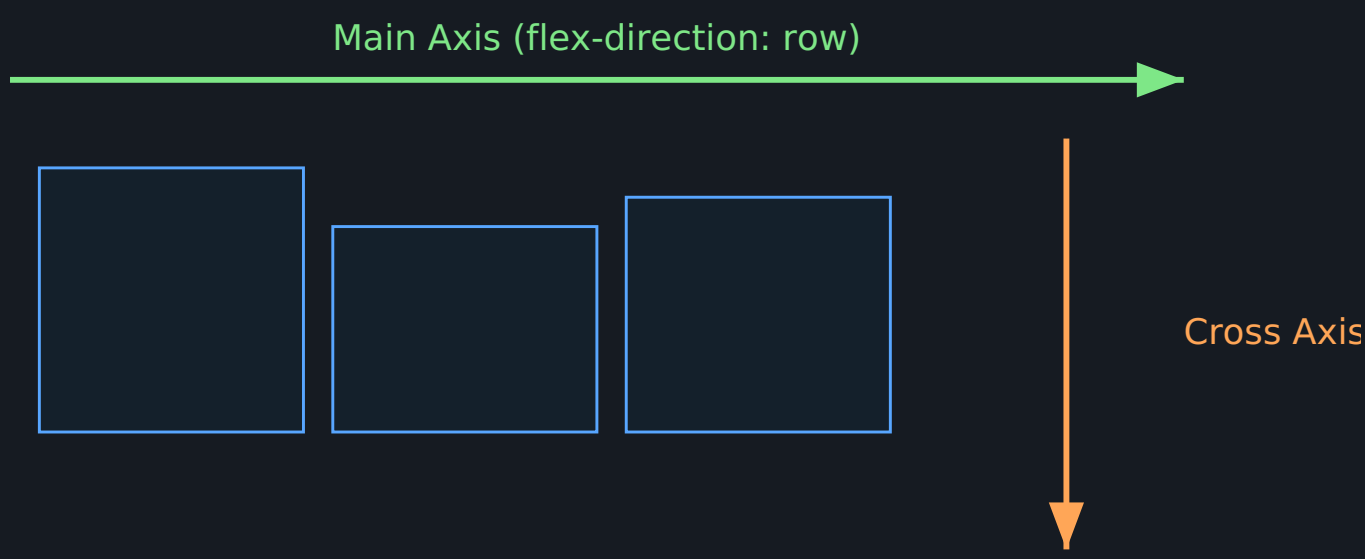
Anchor Size: `anchor-size()` function se positioned element ko anchor ke dimensions ke hisaab se size kar sakte hain — jaise `width: anchor-size()`.

Important Rule: Agar same `anchor-name` multiple bar use ho, to positioned element document mein **last** wale anchor se tether hota hai (jab tak anchor-scope na ho).

8. Flexbox

Flexbox **one-dimensional** layout model hai — items ka group lekar unke liye best layout deta hai. Sidebar + content jaisi responsive patterns ke liye ideal hai.

Main Axis vs Cross Axis



Main axis = flex-direction ki direction; Cross axis = uske perpendicular

Flex items main axis par ek group ki tarah move hote hain. Agar `flex-direction: row` ho, cross axis column direction mein hota hai.

Flex Container Banana

```
.container { display: flex; }
```

Initial (default) behavior:

- Items row mein display hote hain
- Wrap nahi hote (overflow ho sakta hai)
- Grow nahi karte fill karne ke liye
- Start se line-up hote hain

flex-direction

Value	Kaam
row (default)	Row mein, writing mode ke hisaab se
row-reverse	Row, but end se start
column	Column mein
column-reverse	Column, end se start

Accessibility Warning: `row-reverse` / `column-reverse` sirf *visual* order badalte hain, DOM order nahi. Isse keyboard navigation aur screen readers "disconnected" feel karte hain. Zaroorat pade to actual HTML order badlein.

flex-wrap

Default `nowrap` hai — items min-content tak shrink ho kar overflow ho sakte hain. `flex-wrap: wrap` se multiple lines ban sakti hain. Shorthand: `flex-flow: column wrap;`

Growing/Shrinking — flex shorthand

Property	Default	Kaam
flex-grow	0	Extra space mein grow karna
flex-shrink	1	Space kam ho to shrink karna
flex-basis	auto	Base size

```
flex: initial;  /* 0 1 auto - default */
flex: auto;     /* 1 1 auto - content size ke hisaab se grow */
flex: 1;        /* 1 1 0 - sab items equal size */
flex: none;     /* 0 0 auto - inflexible */
flex: 2 1 auto; /* custom grow ratio */
```

flex: auto vs flex: 1: `flex: auto` mein bade items ko zyada space milta hai (content size ke baad distribute hota hai). `flex: 1` mein sab ka basis 0 hota hai, isliye space equally share hoti hai.

order Property

Items ko ordinal groups mein reorder karta hai — lekin yeh sirf visual order badalta hai, DOM order nahi.

Warning: `order` ko document ki galat sequence "fix" karne ke liye use na karein — agar logical order badalni hai to HTML hi badlein.

Alignment Properties

Axis	Space Distribution	Alignment
Main axis	justify-content	—
Cross axis	align-content	align-self / align-items

`justify-content` initial value `flex-start` hai. Values: `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`.

`align-content` ka initial value `stretch` hai (wrapped lines ke liye). `align-self` / `align-items` ka initial value bhi `stretch` hota hai.

Center Karna (Horizontal + Vertical)

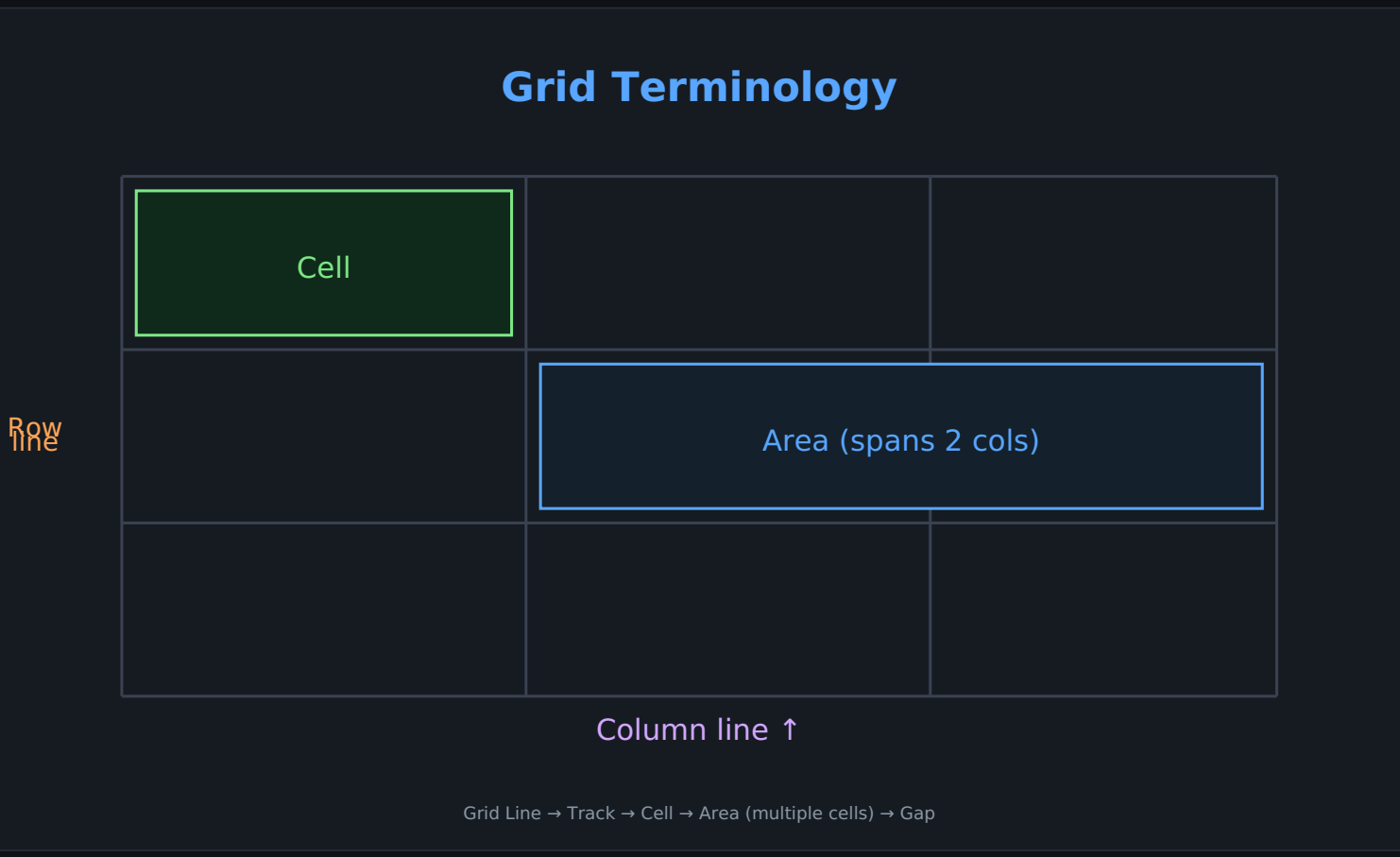
```
.container {
  display: flex;
  justify-content: center; /* main axis */
  align-items: center;     /* cross axis */
}
```

Auto Margins: Flexbox mein `justify-self` nahi hoti (kyun ke items main axis par group ki tarah treat hote hain), lekin `margin-left: auto` se ek item ko group se split kar sakte hain — yeh saara available space absorb kar leta hai.

9. Grid Layout

Grid **two-dimensional** layout model hai — rows aur columns dono ek sath control karta hai. Header + sidebar + content + footer jaisi layouts ke liye perfect.

Grid Terminology



Term	Matlab
Grid Line	Horizontal/vertical lines jo grid banati hain (numbering 1 se shuru)
Grid Track	Do lines ke darmiyan space (row track ya column track)
Grid Cell	Smallest unit — ek row aur ek column ka intersection
Grid Area	Multiple cells mil kar (item span karta hai)
Gap	Tracks ke darmiyan gutter — content place nahi ho sakta yahan

Grid Banana

```
.container {
  display: grid;
  grid-template-columns: 5em 100px 30%;
  grid-template-rows: 200px auto;
  gap: 10px;
}
```

Track Sizing

Method	Kaam
Fixed (px, em, %)	Exact size
min-content	Content ke sabse chote size tak (jaise longest word)

max-content	Poore content ke liye jitni jagah chahiye (wrap nahi hota)
fit-content(N)	max-content jaisa, lekin N tak limit
`fr` unit	Available space ka proportion (jaise flex-grow)

```
grid-template-columns: 1fr 1fr 1fr;      /* teen equal columns */
grid-template-columns: 200px 1fr;        /* fixed + flexible */
grid-template-columns: minmax(0, 1fr);   /* min 0, max 1fr - equal split guarantee */
```

repeat() aur auto-fill/auto-fit

```
grid-template-columns: repeat(12, minmax(0,1fr)); /* 12-column grid system */

/* responsive grid, bina media queries ke */
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

auto-fill vs auto-fit: Agar items kam hon to `auto-fill` empty tracks banata hai (khali jagah rehti hai), lekin `auto-fit` un empty tracks ko collapse kar deta hai aur flexible tracks poori space le lete hain.

Auto-placement aur grid-auto-flow

```
grid-auto-flow: column; /* rows ki bajaye columns mein fill karo */
grid-auto-flow: dense;  /* gaps ko baad ke items se fill karo (order se disconnect ho sakta hai) */
```

Items Place Karna (Line-based)

```
.item {
  grid-column-start: 1;
  grid-column-end: 4;      /* shorthand: grid-column: 1 / 4; */
  grid-row-start: 2;
  grid-row-end: 4;
}
.item2 { grid-column: 1 / -1; } /* poori width, explicit grid ke last line tak */
```

Negative line numbers: Sirf **explicit grid** ke liye kaam karte hain. Implicit grid (auto-created tracks) mein `-1` se end tak reach nahi hota.

Named Lines aur grid-template-areas

```
.container {
  display: grid;
  grid-template-columns: repeat(4,1fr);
  grid-template-areas:
    "header header header header"
    "sidebar content content content"
    "sidebar footer footer footer";
}
header  { grid-area: header; }
.sidebar{ grid-area: sidebar; }
footer  { grid-area: footer; }
```

Rules: har cell filled honi chahiye (khali ke liye `.` use karein); repeat naam se span hota hai; areas rectangular aur connected honi chahiye — warna poori value invalid ho jati hai.

Subgrid

Nested grid item apne parent ki track sizing, line names, aur gap **inherit** kar sakta hai — `grid-template-columns: subgrid` ya `grid-template-rows: subgrid`. Useful jab galleries/cards mein alignment chahiye across nested containers.

Alignment (Grid mein bhi Flexbox jaisi properties)

Axis	Container-level (distribute)	Item-level (move)
Inline (row)	justify-content	justify-self / justify-items
Block (column)	align-content	align-self / align-items

Note: Image jaisi intrinsic-aspect-ratio content ke liye initial `align-self` / `justify-self` value `stretch` ki bajaye `start` hoti hai — taake distortion na ho.

Shorthands

```
/* grid-template: rows / columns, with areas */
.container {
  grid-template:
    "head head head" minmax(150px, auto)
    "sidebar content content" auto
    "sidebar footer footer" auto / 1fr 1fr 1fr;
}
```

Recall Questions — Sab Topics

Khud test karein — pehle answer sochein, phir neeche dekhein.

Selectors

Q1. Class selector aur ID selector mein specificity ka fark kya hai, aur ID kyun kam use karni chahiye?

A: Class (0,1,0), ID (1,0,0). ID zyada specific hai lekin sirf ek element par unique honi chahiye — is se reusability kam hoti hai aur override karna mushkil ho jata hai.

Q2. `[href^='https']` aur `[href$='.com']` mein kya fark hai?

A: `^` = starts-with, `$` = ends-with. Pehla un links ko target karega jo https se start hon, doosra jo .com par end hon.

Q3. Pseudo-class aur pseudo-element mein kya fark hai?

A: Pseudo-class (single colon :) platform state target karti hai (jaise :hover). Pseudo-element (double colon ::) naya virtual element insert karta hai ya element ka specific part target karta hai (jaise ::before).

Specificity

Q4. `a.my-class.another-class[href]:hover` ki specificity kya hogi?

A: (0,4,1) — 2 classes + 1 attribute + 1 pseudo-class = B:4, aur 1 type selector = C:1.

Q5. `:where()` ki specificity hamesha kya hoti hai?

A: (0,0,0) — chahe andar ID ho ya kuch bhi. Isliye basic selectors se bhi override ho jati hai.

Cascade

Q6. Cascade ke 4 stages kya hain, aur inka priority order kya hai?

A: Origin+Importance sabse pehle decide karte hain, phir Specificity, phir Position/Order (last tie-breaker). (Note: topic mein 1-Position, 2-Specificity, 3-Origin, 4-Importance list ki gayi hai jo conceptual order hai.)

Q7. Agar authored CSS mein `!important` ho aur user custom style mein bhi `!important` ho, to konsa jeetega?

A: User custom style ka `!important` jeetega — kyun ke "Local user styles !important" origin authored !important se zyada specific hai.

Inheritance

Q8. Inheritance kis direction mein flow karti hai — parent se child, ya child se parent?

A: Sirf parent se child (neeche ki taraf). Ek child element apni value kabhi parent ko wapis nahi bhejta.

Q9. Kya `margin` aur `padding` default inherit hoti hain?

A: Nahi. Zyada tar layout properties (margin, padding, width, border, display) inherit nahi hotin — sirf typography-related properties (color, font-*, line-height, text-align, etc.) default inherit hoti hain.

Q10. `unset` keyword kis tarah decide karta hai ke woh `inherit` jaisa behave karega ya `initial` jaisa?

A: Agar property default inherit hoti hai (jaise color) to unset = inherit ki tarah kaam karta hai. Agar property default inherit nahi hoti (jaise margin) to unset = initial ki tarah kaam karta hai.

Q11. `revert` aur `unset` / `initial` / `inherit` mein bunyadi fark kya hai?

A: inherit/initial/unset yeh specify karte hain ke value *kya* honi chahiye. revert sirf itna kehta hai ke current origin ke authored styles apply na hon — cascade ek origin peeche chala jata hai (user preference ya user-agent default).

Box Model

Q12. `width: 200px; border: 10px solid; padding: 20px;` **— content-box aur border-box mein actual rendered width kya hogi?**

A: content-box: 260px (200+40+20). border-box: 200px (padding/border andar push ho jate hain).

Q13. Margin box aur outline/box-shadow ka kya relation hai?

A: outline aur box-shadow margin box ki space mein "paint" hote hain — yeh box ka actual size nahi badalte.

Sizing Units

Q14. `em` **aur** `rem` **mein kya fark hai?**

A: em parent element ke font-size se relative hoti hai (nested ho to compound ho sakti hai). rem hamesha root (html) element ke font-size se relative hoti hai — predictable rehti hai.

Q15. line-height ke liye unitless value kyun better hai px se?

A: Kyun ke font-size inherit hoti hai — unitless line-height us ke hisaab se automatically scale ho jati hai, jab ke fixed px har jagah same rehti hai chahe font-size different ho.

Anchor Positioning

Q16. Agar ek hi `anchor-name` **multiple jagah ho, to positioned element kis se tether hoga?**

A: Document mein sabse last waale (aur already laid-out) anchor se — jab tak `anchor-scope` use na ho.

Flexbox

Q17. `flex: auto` **aur** `flex: 1` **mein kya fark hai?**

A: flex: auto (basis: auto) mein bade content wale items ko zyada space milta hai. flex: 1 (basis: 0) mein sab items ka space equally distribute hota hai, content size ignore ho jati hai.

Q18. justify-content kaunsi axis par kaam karta hai, aur align-items kaunsi par?

A: justify-content main axis par (jaise row mein horizontal), align-items cross axis par (row mein vertical).

Q19. Flexbox mein justify-self kyun nahi hoti?

A: Kyun ke flex items main axis par ek group ki tarah treat hote hain — individually split karna concept ke against hai. Iski jagah margin: auto use karte hain.

Grid

Q20. auto-fill aur auto-fit mein kya fark hai (jab items kam hon container se)?

A: auto-fill empty tracks bana kar chorta hai. auto-fit un empty tracks ko collapse kar deta hai, aur flexible (fr) tracks baaki space le lete hain.

Q21. `grid-column: 1 / -1` **hamesha kaam kyun nahi karta?**

A: Negative line numbers sirf explicit grid ke liye valid hain. Agar tracks implicit grid mein ban rahe hon (auto-created), to -1 se end tak reach nahi hoti.

Q22. Subgrid ka main faida kya hai?

A: Nested grid item parent ki track sizing, line names aur gap inherit kar leta hai — is se alignment across nested containers (jaise gallery cards) consistent rehti hai bina manual calculation ke.

Q23. `minmax(0, 1fr)` **kyun use karte hain sirf** `1fr` **ki bajaye?**

A: `fr` unit content ke intrinsic size ke baad space distribute karta hai — is se tracks equal nahi ho sakte. `minmax(0,1fr)` minimum ko 0 kar deta hai, jis se available space accurately equal share hoti hai.

Q24. grid-template-areas mein khali cell kaise define karte hain?

A: `.` (dot) character se — ek ya zyada dots (bina space ke) ek khali cell represent karte hain.